

TrendSparC_MVP 구현 설명서

초기 스캐폴드 구조 및 팀별 작업 가이드

대상 독자

- 브로드밴드 담당자 2인 — sk_broadband 섹터 구현 담당
- 계열사 미정(3번째 계열사) 담당자 2인 — unassigned 섹터 구현 담당
- 도혁님 — 프롬프트 엔지니어 (global/섹터별 system prompt, audience 프로필)
- 성우 — 프로젝트 총괄 (core 오케스트레이션, 계약 스키마 관리, 신규 섹터/오디언스 승인)

※ SK hynix(sk_hynix) 섹터의 README와 프롬프트는 도혁님·성우님이 추후 직접 작성해 넣을 예정입니다. 이 문서와 현재 스캐폴드에는 sk_hynix도 다른 섹터와 동일하게 template_only 뼈대만 들어 있습니다.

경로: Desktop/TrendSparC_MVP (기존 Desktop/TrendSparC 프로젝트와는 별개의 새 프로젝트입니다)

0. 이 문서를 읽는 법

이 문서는 TrendSparC MVP가 무엇이고, 왜 지금 이런 구조로 만들어졌는지, 그리고 폴더 하나하나가 무엇을 위한 자리인지를 설명합니다. 뒷부분(7장)에는 각자 맡은 섹터/역할별로 구체적으로 어떤 파일을 채우면 되는지 실전 가이드가 있으니, 급하면 1~5장은 훑어보고 7장부터 봐도 됩니다.

가장 중요한 원칙 한 가지만 먼저 말씀드리면: 이 스캐폴드는 계약(스키마)과 뼈대만 만들어 둔 상태이고, 실제 수집·분석 로직은 의도적으로 비어 있습니다. 각 담당자가 자신의 섹터 폴더 안에서만 코드를 채우면, 다른 사람의 코드나 core/ 공통 코드를 건드릴 필요가 없도록 설계되어 있습니다.

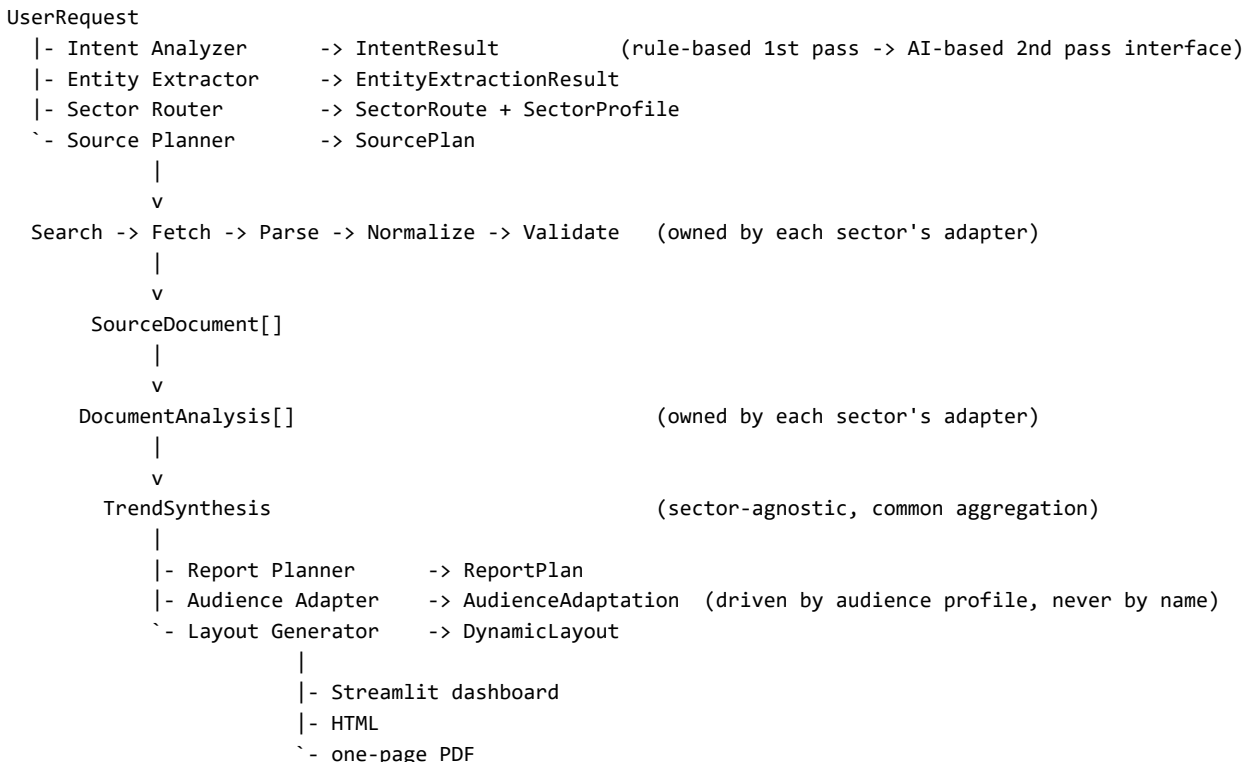
1. 프로젝트 배경

TrendSparC는 SK 계열사(하이닉스 / 브로드밴드 / 3번째 계열사(이름 미정)) 구성원이 사용하는 AI Trend Intelligence 플랫폼입니다. 사용자의 질문을 받아 의도·엔티티·섹터를 판단하고, 섹터별로 데이터를 수집·분석한 뒤, 결과를 보고 받을 대상(청중)에 맞춰 대시보드/HTML/1페이지 PDF로 가공해 제공합니다.

중요 전제: 질문을 입력하는 사람은 항상 우리가 설정한 SK 계열사 소속 직원입니다. "청중(audience)"은 질문자가 아니라 결과 보고서를 누구에게 보여줄지의 타겟입니다 (외부 관계자 / 실무진 / 임원·팀장 / 경영진, 4종, 확장 가능).

2. 전체 파이프라인 아키텍처

사용자 질문 하나가 들어오면 아래 순서로 처리됩니다. 화살표 각 단계는 core/ 아래의 독립된 모듈이 하나씩 담당하고, 모듈 사이에는 항상 정해진 계약(Pydantic 모델)만 오갑니다.



여기서 "섹터별 adapter가 실제 구현 소유"라고 표시된 두 단계(수집~검증, 분석)가 바로 브로드밴드 팀, 미정 계열사 팀, 그리고 나중에 하이닉스 팀이 각자 채워 넣을 부분입니다.

3. 핵심 설계 원칙 (반드시 지킬 것)

3.1 계약 기반 블록 격리 (Contract Isolation)

모든 블록 간 경계는 `common/` 또는 `sectors/< >/contracts/`에 정의된 Pydantic 모델로만 통신합니다. 한 블록(예: `sk_broadband`의 `analyzer`) 내부를 수정해도, 입출력 계약이 그대로라면 다른 블록(다른 섹터, `core`, `audience` 등) 코드는 전혀 수정할 필요가 없습니다.

따라서 섹터 이름이나 청중 이름으로 `if/else` 분기하는 코드는 금지입니다. 항상 등록된 `profile.json / audience` 프로필을 코드가 알아서 찾아 읽는 방식으로만 동작해야 합니다.

3.2 실패 추적 가능성 (Failure Traceability)

각 파이프라인 단계는 실패 시 "어느 단계(stage id)에서, 어떤 사유로 실패했는지"를 구조화된 형태(`StageTrace`)로 남깁니다. 요청 1건마다 단계별 실행 기록이 남기 때문에, 문제가 생기면 로그를 뒤지지 않고도 정확히 어느 단계가 원인인지 바로 알 수 있습니다.

4. 전체 폴더 구조

[illegible]

5. 폴더별 상세 설명

5.1 루트 파일

- `main.py` — CLI 진입점. 질문 하나를 넣으면 파이프라인을 dry-run으로 실행하고, 단계별 실행 기록(trace)과 중간 결과를 JSON으로 출력합니다.
- `requirements.txt` — pydantic, pytest, pyyaml 세 개뿐입니다. 아직 실제 수집/분석 로직이 없어서 외부 API 클라이언트 라이브러리는 넣지 않았습니다.
- `conftest.py` — pytest가 프로젝트 루트를 import 경로에 잡아주기 위한 설정 파일입니다(직접 건드릴 일은 거의 없음).
- `.env.example` — 지금은 쓰이지 않습니다. 나중에 2차 AI 의도분석에 API 키가 필요해지면 그때 채웁니다.

5.2 common/ — 공통 계약과 실패 모델

모든 팀이 공통으로 참조만 하고, 함부로 고치지 않는 영역입니다. 필드를 추가해야 할 일이 생기면 성우님과 상의 후 진행합니다.

- `contracts.py` — `UserRequest`, `IntentResult`, `EntityExtractionResult`, `SectorProfile`, `SectorRoute`, `SourcePlan`, `SourceDocument`, `DocumentAnalysis`, `TrendSynthesis`, `ReportPlan`, `AudienceAdaptation`, `DynamicLayout` 등 모든 블록 경계에서 쓰이는 Pydantic 모델 12종이 들어 있습니다.
- `errors.py` — `StageStatus(ok/template_only/failed/skipped)`, `StageTrace`, `PipelineStageError`. 어떤 섹터든 아직 구현이 안 된 단계는 반드시 `PipelineStageError`를 `reason="template_only: ..."` 형태로 던지도록 되어 있습니다.

5.3 core/ — 오케스트레이션 & 공통 인프라

여기는 섹터에 상관없이 항상 동일하게 동작하는 로직입니다. 특정 섹터를 위한 코드는 절대 여기에 들어가지 않습니다.

intent/

- `rule_based.py` — “비교/전망/리스크/트렌드/현황” 같은 키워드로 1차 분류하는 항상 동작하는 규칙 기반 분류기.
- `ai_based.py` — 2차 AI 기반 정밀 분류를 위한 인터페이스만 존재. 지금은 API 키가 없으므로 규칙기반 결과를 그대로 반환합니다. 실제 AI 호출 코드는 아직 채우지 않았습니다.

entity/extractor.py

- 질문 문장에서 기업명·기술명·키워드를 뽑아내는 공용 로직(섹터별 특수 로직 아님).

sector_router/router.py

- `sectors/*/profile.json`을 실행 시점에 스캔해서 섹터 레지스트리를 만듭니다. 섹터 이름은 코드에 절대 하드코딩되어 있지 않습니다 — 폴더를 추가/삭제하면 라우팅 결과가 즉시 바뀝니다.
- 요청에 명시된 섹터가 등록돼 있지 않으면 상태를 “unsupported”로 명확히 반환합니다.

source_planner/planner.py

- `sources/registry/< id>/`를 읽어 `SourcePlan`을 만듭니다. 지금은 모든 레지스트리가 비어 있어 “template_only” 메모가 남습니다.

synthesis/synthesizer.py

- 섹터별 analyzer가 만든 DocumentAnalysis 리스트를 단순 집계(개수 세기, 포인트 합치기)만 합니다. 없는 인사이트를 지어내지 않습니다.

report_planner/planner.py, layout_generator/generator.py

- 청중(audience) 프로필을 읽어 보고서 섹션 구성과 포맷(dashboard/html/pdf)을 정하고, 실제 콘텐츠 없이 블록 배치만 담은 DynamicLayout을 만듭니다.

request_pipeline/pipeline.py

- 위 모든 단계를 정해진 순서로 호출하고, 각 단계마다 StageTrace를 하나씩 남기는 지휘자입니다. dry_run=True(기본값)일 때는 섹터 adapter(수집~분석) 코드를 아예 호출하지 않고 "skipped"로 표시합니다. force_fail_stage 파라미터로 특정 단계를 일부러 실패시켜 테스트할 수 있습니다.

5.4 sectors/ — 섹터별 실제 구현이 들어갈 자리

네 개 섹터(sk_hynix, sk_broadband, unassigned, general) 모두 동일한 구조를 가지고 있고, 현재는 전부 status: "template_only" 입니다. 각 섹터 폴더 구성은 다음과 같습니다.

파일/폴더	역할
README.md	이 섹터의 상태와 담당 범위 설명
profile.json	sector_id, aliases, keywords 등 라우팅에 쓰이는 메타데이터
prompts/system_prompt.md	이 섹터 전용 분석 프롬프트(현재 빈 템플릿)
contracts/__init__.py	공통 계약 재수출 자리(섹터 전용 필드 추가 시 여기 확장)
adapter/collector/	Search/Fetch 담당 — 실제 수집 코드가 들어갈 자리
adapter/processor/	Parse/Normalize 담당
adapter/validator/	Validate 담당
adapter/analyzer/	DocumentAnalysis 생성 담당
adapter/reporter/	섹터 전용 리포트 확장 포인트(선택적)

지금 각 adapter/<역할>/__init__.py 안의 함수는 전부 아래처럼 PipelineStageError를 던지지만 합니다.

```
def collect(source_plan):
    raise PipelineStageError(
        stage="sectors.sk_broadband.adapter.collector",
        reason="template_only: sector adapter not implemented",
    )
```

각 담당 팀이 할 일은 이 함수 내용을 실제 로직으로 교체하는 것입니다. 함수 시그니처(입력/출력 타입)만 그대로 유지하면 core나 다른 섹터 코드는 전혀 건드릴 필요가 없습니다.

5.5 sources/registry/

섹터별로 “어떤 소스를 신뢰하고 쓸 것인지” 등록해 두는 곳입니다. 지금은 {"sources": []} 빈 상태입니다. 등록되지 않은 소스에는 임의로 신뢰도(reliability_tier)를 부여하면 안 됩니다 — 반드시 여기에 먼저 등록해야 합니다.

5.6 audience/ — 청중별 표현 방식

audience/profiles/ 아래 4개 파일(external, practitioner, executive, management)은 청중별 톤·상세도·포맷을 정의합니다. audience/adapter.py와 core/report_planner는 이 프로필 파일만 읽고 동작하며, 청중 이름으로 코드 분기를 하지 않습니다.

※ 현재 4개 프로필 파일 내용은 원 기획안에 확정 문구가 포함되어 있지 않아 제가 임시 초안으로 채워 두었습니다(파일 상단에 “DRAFT — not yet confirmed content” 표기). 도혁님이 실제 확정된 톤/문구로 교체해 주셔야 합니다.

5.7 prompts/global_system_prompt.md

섹터·청중과 무관하게 항상 적용되는 공통 원칙 문서입니다: 근거 없는 추측 금지, 출처 필수, 사실과 해석 구분, 미등록 소스에 임의 신뢰도 부여 금지. 섹터별 system_prompt.md는 이 문서의 원칙을 전제로 내용을 추가하는 것이지, 절대 완화하거나 덮어쓰지 않습니다.

5.8 reporting/{dashboard_streamlit, html, pdf}/

각각 render(layout) 함수 자리만 있고, 지금은 "template_only: renderer not implemented" 메시지만 반환합니다. 실제 Streamlit 화면/HTML/PDF 렌더링은 이후 단계에서 구현합니다.

5.9 ~ 5.12 examples/, tests/, storage/, docs/

- examples/ — 파이프라인에 넣어볼 수 있는 예시 요청 JSON 1건.
- tests/ — pytest 스위트. 전체 dry-run 통과, 미등록 섹터 unsupported 처리, 섹터 폴더 추가/삭제 시 core 무변경 확인, 강제 실패 추적 등을 검증합니다.
- storage/ — 실제 요청 실행 결과가 나중에 저장될 자리(현재는 비어 있음).
- docs/architecture.md — 이 문서의 2~3장 내용을 코드 저장소 안에도 남겨 둔 버전.

6. 실행 및 테스트 방법

설치

```
cd TrendSparC_MVP
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

dry-run 파이프라인 실행

```
python main.py --question "SK hynix HBM market outlook?" --audience practitioner
python main.py --request-file examples/requests/sample_request.json
python main.py --question "..." --sector sk_totally_made_up # -> unsupported
python main.py --question "..." --force-fail-stage intent # -> forced failure trace
```

※ 실제 사용 시에는 --question 값에 한글 질문을 그대로 넣어도 됩니다 (예: "SK하이닉스 HBM 시장 전망은?"). 위 예시는 문서 편집 편의상 영문으로 표기했습니다.

테스트

```
pytest
```

현재 8개 테스트가 모두 통과하는 상태입니다.

7. 담당자별 작업 가이드

각자 자신의 섹터 폴더 “안에서만” 작업하면 됩니다. core/, common/, 다른 섹터 폴더는 건드릴 필요가 없습니다(공용 계약 확장이 꼭 필요하다면 성우님과 먼저 상의).

7.1 브로드밴드 담당자 (2인) — sectors/sk_broadband/

- profile.json의 aliases/keywords를 실제 업무에 맞게 보강하세요(현재는 예시 수준: “SK Broadband, SK브로드밴드, 브로드밴드 / 5G, 6G, 통신, 네트워크”).
- README.md에 이 섹터가 실제로 다룰 데이터 범위(예: 통신 시장, 네트워크 인프라, 정책 등)를 적어 주세요.
- adapter/collector, processor, validator, analyzer 안의 stub 함수를 실제 로직으로 교체하세요. 함수 입력/출력 타입(SourcePlan → SourceDocument 리스트 → DocumentAnalysis 리스트)만 지키면 됩니다.
- 사용할 소스가 정해지면 sources/registry/sk_broadband/sources.json에 등록하세요. 등록 전에는 임의로 신뢰도를 매기지 마세요.
- adapter/reporter는 섹터 전용 리포트 확장이 필요할 때만 채우면 되는 선택 항목입니다.

7.2 미정 계열사 담당자 (2인) — sectors/unassigned/

- 현재 unassigned는 aliases/keywords가 비어 있는 완전 빈 템플릿입니다. 계열사명이 정해지면 폴더명을 unassigned에서 실제 이름(예: sk_xxx)으로 바꾸고, profile.json의 sector_id와 aliases/keywords, README.md, adapter 내부 stub의 stage 이름(sectors.unassigned... → sectors.<확정명>...)을 함께 갱신해 주세요.
- 폴더명을 바꾸는 것만으로 core/sector_router가 자동으로 새 이름을 인식합니다 — core/ 코드는 손댈 필요가 없습니다.
- 그 외 작업 방식은 7.1 브로드밴드 가이드와 동일합니다(profile 보강 → adapter 구현 → 소스 레지스트리 등록).

7.3 도혁님 — 프롬프트 엔지니어

- prompts/global_system_prompt.md — 섹터 무관 공통 원칙. 지금 내용(근거 없는 추측 금지, 출처 필수 등)을 검토/보강해 주세요.
- sectors/<섹터>/prompts/system_prompt.md — 섹터별 분석 프롬프트. 지금은 전부 빈 템플릿이며, 각 섹터가 실제로 무엇을 분석 범위/제외 범위로 삼을지, 섹터 전용 유의사항은 무엇인지를 채워 주셔야 합니다.
- audience/profiles/{external, practitioner, executive, management}.md — 5.6절에서 말씀드린 대로 지금 내용은 전부 임시 초안입니다. 실제 확정된 톤/상세도/포맷 기준으로 교체해 주세요(파일 상단 YAML 부분의 tone, detail_level, focus, format_preference 값과 본문 설명 모두).
- sk_hynix 섹터의 README.md와 prompts/system_prompt.md는 성우님과 함께 추후 직접 채워 넣으실 예정입니다 — 지금은 다른 섹터와 동일한 빈 템플릿 상태로 남겨 두었습니다.

7.4 성우 — 프로젝트 총괄

- common/contracts.py — 신규 필드가 필요하다는 요청이 오면 여기서 검토/추가. 필드 추가는 기존 계약을 깨지 않는 선(옵셔널 필드 추가 등)에서 진행.
- core/ — 오케스트레이션 로직 관리. 새 파이프라인 단계가 필요해지면 여기서 설계.
- 신규 섹터/청중 등록 검토 — 각 팀이 만든 profile.json / audience 프로필이 규칙(하드코딩 금지, 등록 기반 라우팅)을 지키는지 확인.

- sk_hynix 섹터 README/프롬프트를 도혁님과 함께 직접 작성.

8. 신규 섹터 / 신규 청중 추가하는 법(참고용)

새 섹터 추가

- sectors/<새섹터id>/ 폴더를 다른 섹터와 같은 구조로 생성
- profile.json에 sector_id, display_name, status("template_only"), aliases, keywords 작성
- adapter/{collector,processor,validator,analyzer,reporter}에 stub 또는 실제 구현 작성
- sources/registry/<새섹터id>/sources.json 생성(빈 배열로 시작 가능)
- core/ 코드는 수정할 필요 없음 — sector_router가 다음 실행부터 자동 인식

새 청중 추가

- audience/profiles/<새청중id>.md 생성(YAML frontmatter: display_name, tone, detail_level, focus, format_preference + 본문 설명)
- core/report_planner, audience/adapter는 코드 변경 없이 새 프로필을 자동으로 읽어 사용

9. 공통 주의사항

- 섹터/청중 이름으로 if/else 분기하는 코드를 작성하지 마세요 — 항상 profile.json / audience 프로필을 통해서만 동작이 달라져야 합니다.
- 실제 분석 결과가 없는 상태에서 그럴듯한 가짜 데이터를 채워 넣지 마세요. 구현이 안 됐으면 PipelineStageError(template_only)로 명확히 표시하세요.
- 등록되지 않은 소스에 임의로 reliability_tier를 부여하지 마세요.
- 사용자 요청 없이 외부 API(예: 뉴스 크롤러, LLM API)를 호출하는 코드를 넣지 마세요 — 이번 단계 범위 밖입니다.
- 스케줄러/백그라운드 폴링/정기 캐시 갱신은 이 프로젝트 범위가 아닙니다.

문의사항은 성우님에게 — 계약(스키마) 변경이 필요한 경우 반드시 사전 협의 후 진행해 주세요.